05-비동기-IO-최적화

개요

Monad는 io_uring 기반의 비동기 I/O 시스템을 통해 초고속 디스크 접근을 구현합니다. 이 문서는 `category/async` 디렉토리의 비동기 I/O 최적화 기법을 상세히 분석합니다.

주요 최적화 기법:

- io_uring을 통한 커널 바이패스 I/O
- 배치 제출(Batch Submission)로 시스템 콜 최소화
- 사전 등록된 버퍼 풀(Buffer Pool) 관리
- Sender/Receiver 패턴을 통한 타입 안전한 비동기 작업 체인
- 우선순위 기반 I/O 스케줄링

관련 파일:

- `category/async/io.hpp/cpp` : AsyncIO 메인 클래스
- `category/async/io_senders.hpp` : Sender 구현
- `category/async/connected_operation.hpp` : 연결된 작업 관리
- `category/async/storage_pool.cpp` : 스토리지 풀 관리

---

1. io_uring 통합 분석

1.1 AsyncIO 클래스 구조

`AsyncIO` 클래스는 io_uring을 래핑하여 비동기 I/O를 제공하는 핵심 컴포넌트입니다.

```
// category/async/io.hpp
class AsyncIO final
{
private:
pid_t  const owning_tid_; // 소유 스레드 ID
class storage_pool *storage_pool_{nullptr}; // 스토리지 풀
monad::io::Ring &uring_; // 읽기용 io_uring
monad::io::Ring *wr_uring_{nullptr}; // 쓰기 전용 io_uring (옵션)
monad::io::Buffers &rwbuf_; // 버퍼 관리자
monad::io::BufferPool rd_pool_; // 읽기 버퍼 풀
monad::io::BufferPool wr_pool_; // 쓰기 버퍼 풀
IORecord records_; // I/O 통계
unsigned concurrent_read_io_limit_{0}; // 동시 읽기 제한
public:
// 읽기 요청 제출
size_t  submit_read_request(
std::span<std::byte> buffer,
chunk_offset_t offset,
erased_connected_operation *uring_data);
// 쓰기 요청 제출
void  submit_write_request(
std::span<std::byte const> buffer,
```

```
chunk_offset_t offset,
erased_connected_operation *uring_data);
// 완료 폴링
size_t  poll_blocking(size_t  count = 1);
size_t  poll_nonblocking(size_t  count = size_t(-1));
};
```

핵심 특징:

- 스레드별 인스턴스: 각 스레드는 독립적인 AsyncIO 인스턴스 보유
- 분리된 읽기/쓰기 링: 쓰기 전용 io_uring으로 읽기와 쓰기 분리
- 버퍼 풀 관리: 사전 할당된 DMA 정렬 버퍼 재사용

1.2 읽기 요청 제출 (submit_read_request)

읽기 요청은 io_uring의 SQE(Submission Queue Entry)를 통해 제출됩니다.

```
// category/async/io.cpp
void AsyncIO::submit_request_(
std::span<std::byte> buffer,
chunk_offset_t chunk_and_offset,
void  *uring_data,
enum erased_connected_operation::io_priority prio)
{
MONAD_DEBUG_ASSERT((chunk_and_offset.offset & (DISK_PAGE_SIZE - 1)) == 0);
MONAD_DEBUG_ASSERT(buffer.size() <= READ_BUFFER_SIZE);
```

```
// 제출 큐가 가득 차면 완료 폴링
poll_uring_while_submission_queue_full_();
// SQE 획득
struct io_uring_sqe *sqe = io_uring_get_sqe(&uring_.get_ring());
MONAD_ASSERT(sqe);
// 읽기 작업 준비 (사전 등록된 버퍼 사용)
auto  const &ci = seq_chunks_[chunk_and_offset.id];
io_uring_prep_read_fixed(
sqe,
ci.io_uring_read_fd, // 파일 디스크립터
buffer.data(), // 버퍼
static_cast<unsigned  int>(buffer.size()),
ci.ptr->read_fd().second + chunk_and_offset.offset, // 오프셋
0); // 버퍼 인덱스
// 사전 등록된 파일 디스크립터 사용
sqe->flags |= IOSQE_FIXED_FILE;
// 우선순위 설정
switch (prio) {
case erased_connected_operation::io_priority::highest:
sqe->ioprio = IOPRIO_PRIO_VALUE(IOPRIO_CLASS_RT, 7); // 실시간 최고 우선순위
break;
case erased_connected_operation::io_priority::idle:
sqe->ioprio = IOPRIO_PRIO_VALUE(IOPRIO_CLASS_IDLE, 0); // 유휴 우선순위
```

```
break;
default:
sqe->ioprio = 0; // 기본 우선순위
break;
}
// 사용자 데이터 설정 (완료 시 콜백용)
io_uring_sqe_set_data(sqe, uring_data);
// 제출 (배치 처리)
MONAD_ASYNC_IO_URING_RETRYABLE(io_uring_submit(&uring_.get_ring()));
}
```

최적화 포인트:

1. **Fixed File**: `IOSQE_FIXED_FILE` 플래그로 사전 등록된 파일 디스크립터 사용
2. **Fixed Buffer**: `io_uring_prep_read_fixed` 로 사전 등록된 버퍼 사용
3. **우선순위 제어**: I/O 우선순위를 통한 중요한 작업 우선 처리
4. **배치 제출**: 여러 요청을 모아서 한 번에 제출

1.3 쓰기 요청 제출 (submit_write_request)

쓰기 요청은 전용 io_uring 링을 통해 순차적으로 처리됩니다.

```
// category/async/io.cpp
void AsyncIO::submit_request_(
std::span<std::byte const> buffer,
chunk_offset_t chunk_and_offset,
```

```
void  *uring_data,
enum erased_connected_operation::io_priority prio)
{
MONAD_ASSERT(!rwbuf_.is_read_only());
MONAD_DEBUG_ASSERT((chunk_and_offset.offset & (DISK_PAGE_SIZE - 1)) == 0);
auto  const &ci = seq_chunks_[chunk_and_offset.id];
auto offset = ci.ptr->write_fd(buffer.size()).second;
// 쓰기 전용 링 선택 (없으면 읽기 링 사용)
auto *const wr_ring =
(wr_uring_ != nullptr) ? &wr_uring_->get_ring() : &uring_.get_ring();
struct io_uring_sqe *sqe = io_uring_get_sqe(wr_ring);
MONAD_ASSERT(sqe);
// 쓰기 작업 준비
io_uring_prep_write_fixed(
sqe,
ci.io_uring_write_fd,
buffer.data(),
static_cast<unsigned  int>(buffer.size()),
offset,
wr_ring == &uring_.get_ring());
sqe->flags |= IOSQE_FIXED_FILE;
// 쓰기 전용 링에서는 순차 실행 보장
if (wr_ring != &uring_.get_ring()) {
```

```
sqe->flags |= IOSQE_IO_DRAIN; // 이전 작업 완료 후 실행
}
// 우선순위 설정
switch (prio) {
case erased_connected_operation::io_priority::highest:
sqe->ioprio = IOPRIO_PRIO_VALUE(IOPRIO_CLASS_RT, 7);
break;
case erased_connected_operation::io_priority::idle:
sqe->ioprio = IOPRIO_PRIO_VALUE(IOPRIO_CLASS_IDLE, 0);
break;
default:
sqe->ioprio = 0;
break;
}
io_uring_sqe_set_data(sqe, uring_data);
MONAD_ASYNC_IO_URING_RETRYABLE(io_uring_submit(wr_ring));
}
```

쓰기 최적화 전략:

1. **전용 링**: 쓰기 전용 io_uring으로 읽기와 분리
2. **순차 실행**: `IOSQE_IO_DRAIN` 으로 쓰기 순서 보장
3. **Append-Only**: 순차 쓰기로 SSD 수명 연장
4. **페이싱**: SSD 속도에 맞춰 쓰기 속도 조절

1.4 완료 폴링 (poll_uring_)

io_uring의 CQE(Completion Queue Entry)를 폴링하여 완료된 작업을 처리합니다.

```
// category/async/io.cpp
size_t AsyncIO::poll_uring_(bool  blocking, unsigned  poll_rings_mask)
{
auto h = detail::AsyncIO_per_thread_state().enter_completions();
struct io_uring_cqe *cqe = nullptr;
auto *const other_ring = &uring_.get_ring();
auto *const wr_ring =
(wr_uring_ != nullptr) ? &wr_uring_->get_ring() : nullptr;
// 쓰기 링 완료 확인
if (wr_ring != nullptr && records_.inflight_wr > 0) {
ring = wr_ring;
if (wr_uring_->must_call_uring_submit() ||
!!(wr_ring->flags & IORING_SETUP_IOPOLL)) {
// 폴링 모드에서는 명시적으로 submit 호출
MONAD_ASYNC_IO_URING_RETRYABLE(io_uring_submit(wr_ring));
}
io_uring_peek_cqe(wr_ring, &cqe);
}
// 읽기 링 완료 확인
if (cqe == nullptr) {
```

```
ring = other_ring;
if (blocking && inflight_ts == 0 && records_.inflight_wr == 0) {
// 블로킹 대기
MONAD_ASYNC_IO_URING_RETRYABLE(io_uring_wait_cqe(ring, &cqe));
} else {
// 논블로킹 확인
io_uring_peek_cqe(ring, &cqe);
}
}
if (cqe != nullptr) {
// 사용자 데이터 추출
void *data = io_uring_cqe_get_data(cqe);
state = reinterpret_cast<erased_connected_operation *>(data);
// 결과 처리
res = (cqe->res < 0)
? result<size_t>(posix_code(-cqe->res))
: result<size_t>(cqe->res);
// CQE 소비
io_uring_cqe_seen(ring, cqe);
// 레이턴시 측정
if (capture_io_latencies_) {
state->elapsed =
std::chrono::steady_clock::now() - state->initiated;
```

```
}
}
// 완료 콜백 호출
if (state->is_read()) {
--records_.inflight_rd;
} else  if (state->is_write()) {
--records_.inflight_wr;
}
state->completed(std::move(res));
return  completions.size();
}
```

폴링 최적화:

1. **Eager Completions**: 가능한 모든 완료를 한 번에 처리
2. **IOPOLL 모드**: 커널 폴링으로 레이턴시 감소
3. **배치 처리**: 여러 완료를 벡터에 모아서 처리
4. **레이턴시 측정**: 옵션으로 I/O 레이턴시 추적

---

2. 버퍼 관리 전략 분석

2.1 버퍼 풀 구조

AsyncIO는 읽기와 쓰기를 위한 별도의 버퍼 풀을 관리합니다.

```
// category/async/io.hpp
```

```
class AsyncIO final
{
private:
monad::io::BufferPool rd_pool_; // 읽기 버퍼 풀
monad::io::BufferPool wr_pool_; // 쓰기 버퍼 풀
public:
// 버퍼 크기 상수
static constexpr size_t READ_BUFFER_SIZE = 8 * DISK_PAGE_SIZE; // 32KB
static constexpr size_t WRITE_BUFFER_SIZE = 8 * 1024 * 1024; // 8MB
// 읽기 버퍼 획득
read_buffer_ptr get_read_buffer(size_t bytes) noexcept
{
MONAD_DEBUG_ASSERT(bytes <= READ_BUFFER_SIZE);
unsigned char *mem = rd_pool_.alloc();
if (mem == nullptr) {
// 버퍼 고갈 시 완료 폴링하여 버퍼 회수
mem = poll_uring_while_no_io_buffers_(false);
}
return read_buffer_ptr(
(std::byte *)mem,
detail::read_buffer_deleter(this));
}
// 쓰기 버퍼 획득
```

```
write_buffer_ptr get_write_buffer() noexcept
{
unsigned  char *mem = wr_pool_.alloc();
if (mem == nullptr) {
mem = poll_uring_while_no_io_buffers_(true);
}
return  write_buffer_ptr(
(std::byte *)mem,
detail::write_buffer_deleter(this));
}
// 버퍼 해제
void  do_free_read_buffer(std::byte *b) noexcept
{
#ifndef NDEBUG
memset((void *)b, 0xff, READ_BUFFER_SIZE); // 디버그 모드에서 메모리 오염 감지
#endif
rd_pool_.release((unsigned  char *)b);
}
void  do_free_write_buffer(std::byte *b) noexcept
{
#ifndef NDEBUG
static_assert(WRITE_BUFFER_SIZE >= CPU_PAGE_SIZE);
memset((void *)b, 0xff, CPU_PAGE_SIZE);
```

```
#endif
wr_pool_.release((unsigned  char *)b);
}
};
```

버퍼 크기 설계:

- 읽기 버퍼: 32KB (8 × 4KB 페이지)
- 쓰기 버퍼: 8MB (대용량 배치 쓰기)
- 읽기는 작고 빠르게, 쓰기는 크게 배치 처리

2.2 DMA 정렬 및 사전 등록

버퍼는 DMA(Direct Memory Access)를 위해 정렬되고 io_uring에 사전 등록됩니다.

```
// category/async/io.hpp
// DMA 정렬 버퍼 구조
template <class ConnectedOperationType>
struct registered_io_buffer_with_connected_operation
{
// DMA 페이지 크기로 정렬
alignas(DMA_PAGE_SIZE) std::byte write_buffer[WRITE_BUFFER_SIZE];
// 연결된 작업 상태 (가변 크기 배열)
ConnectedOperationType state[0];
constexpr  registered_io_buffer_with_connected_operation() {}
```

```
};
```

DMA 정렬의 이점:

1. **제로 카피**: 커널이 버퍼를 직접 사용 가능
2. **캐시 효율성**: 캐시 라인 정렬로 false sharing 방지
3. **하드웨어 가속**: NVMe 컨트롤러가 직접 메모리 접근

2.3 버퍼 고갈 처리

버퍼가 고갈되면 완료 폴링을 통해 버퍼를 회수합니다.

```
// category/async/io.cpp
unsigned char *AsyncIO::poll_uring_while_no_io_buffers_(bool is_write)
{
// 새로운 I/O 시작 방지
auto h = detail::AsyncIO_per_thread_state().enter_completions();
for (;;) {
// 진행 중인 I/O가 없으면 치명적 오류
if (0 == io_in_flight()) {
std::cerr
<< "FATAL: no i/o buffers remaining. is_write = " << is_write
<< " within_completions_count = "
<< detail::AsyncIO_per_thread_state().within_completions_count
<< std::endl;
MONAD_ABORT("no i/o buffers remaining");
```

```
}

// 완료 폴링 (쓰기 또는 읽기 링만)

poll_uring_(true, is_write ? 1 : 2);

// 버퍼 재시도

auto *mem = (is_write ? wr_pool_ : rd_pool_).alloc();

if (mem != nullptr) {

return mem;

}

}

}
```

버퍼 고갈 방지 전략:

1. **블로킹 폴링**: 버퍼가 해제될 때까지 대기
2. **선택적 폴링**: 쓰기/읽기 링만 선택적으로 폴링
3. **데드락 감지**: 진행 중인 I/O가 없으면 즉시 중단

2.4 RAII 기반 버퍼 관리

버퍼는 스마트 포인터를 통해 자동으로 관리됩니다.

```
// category/async/io.hpp

namespace detail

{

struct read_buffer_deleter

{
```

```
AsyncIO *parent_;

explicit  read_buffer_deleter(AsyncIO *parent) : parent_(parent) {}

void  operator()(std::byte *b)

{

parent_->do_free_read_buffer(b);

}

};

struct write_buffer_deleter

{

AsyncIO *parent_;

explicit  write_buffer_deleter(AsyncIO *parent) : parent_(parent) {}

void  operator()(std::byte *b)

{

parent_->do_free_write_buffer(b);

}

};

}


using read_buffer_ptr = std::unique_ptr<std::byte, detail::read_buffer_deleter>;

using write_buffer_ptr = std::unique_ptr<std::byte, detail::write_buffer_deleter
```

RAII의 장점:

1. **자동 해제**: 스코프를 벗어나면 자동으로 버퍼 반환

2. **예외 안전성**: 예외 발생 시에도 버퍼 누수 방지

3. **명시적 소유권**: 버퍼 소유권이 명확함

2.5 버퍼 통계 추적

AsyncIO는 버퍼 사용 통계를 추적합니다.

```
// category/async/io.hpp

struct IORecord

{

unsigned inflight_rd{0}; // 진행 중인 읽기

unsigned inflight_rd_scatter{0}; // 진행 중인 scatter 읽기

unsigned inflight_wr{0}; // 진행 중인 쓰기

unsigned inflight_tm{0}; // 진행 중인 타이머

std::atomic<unsigned> inflight_ts{0}; // 진행 중인 스레드 안전 작업

unsigned max_inflight_rd{0}; // 최대 동시 읽기

unsigned max_inflight_rd_scatter{0};

unsigned max_inflight_wr{0};

unsigned nreads{0}; // 총 읽기 횟수

unsigned reads_retried{0}; // 재시도된 읽기

};
```

통계 활용:

- 성능 모니터링
- 버퍼 풀 크기 조정

- 병목 지점 식별

---

3. Sender/Receiver 패턴 분석

3.1 Sender/Receiver 개념

Monad는 C++23 표준의 Sender/Receiver 패턴을 기반으로 타입 안전한 비동기 작업을 구현합니다.

핵심 개념:

- **Sender**: 비동기 작업을 나타내는 객체 (작업 정의)
- **Receiver**: 작업 완료 시 호출될 콜백 (완료 처리)
- **Connected Operation**: Sender와 Receiver를 연결한 실행 가능한 상태

```
┌──────────┐ connect() ┌──────────────────┐ start() ┌──────────┐
| Sender | ──────────────> | Connected | ──────────────> | Running |
| | | Operation | | |
└──────────┘ └──────────────────┘ └──────────┘
|
| completed()
▼
┌──────────┐
|Receiver |
|Callback |
└──────────┘
```

3.2 Sender 구현

3.2.1 read_single_buffer_sender

단일 버퍼 읽기를 위한 Sender입니다.

```
// category/async/io_senders.hpp

class read_single_buffer_sender

{

public:

using buffer_type = filled_read_buffer;

using result_type = result<std::reference_wrapper<buffer_type>>;

static  constexpr operation_type my_operation_type = operation_type::read;

private:

chunk_offset_t offset_; // 읽을 위치

buffer_type buffer_; // 버퍼

public:

constexpr  read_single_buffer_sender(

chunk_offset_t offset, size_t  bytes_to_read)

: offset_(offset)

, buffer_(bytes_to_read)

{

}

// 작업 시작

result<void> operator()(erased_connected_operation *io_state) noexcept
```

```
{
if (!buffer_) {
// 버퍼 할당
buffer_.set_read_buffer(
io_state->executor()->get_read_buffer(buffer_.size()));
}
// 읽기 요청 제출
size_t bytes_transferred = io_state->executor()->submit_read_request(
buffer_.to_mutable_span(), offset_, io_state);
if (bytes_transferred != size_t(-1)) {
// 즉시 완료 (캐시 히트 등)
return  make_status_code(
sender_errc::initiation_immediately_completed,
bytes_transferred);
}
return  success();
}
// 완료 처리
result_type completed(
erased_connected_operation *,
result<size_t> bytes_transferred) noexcept
{
if (bytes_transferred.has_error()) {
```

```
fprintf(stderr,
"ERROR: Read single buffer of %zu bytes from chunk %u offset "
"%llu failed with error '%s'\n",
buffer().size(), offset().id,
file_offset_t(offset().offset),
bytes_transferred.assume_error().message().c_str());
}
BOOST_OUTCOME_TRY(auto &&count, std::move(bytes_transferred));
buffer_.set_bytes_transferred(count);
return  success(std::ref(buffer_));
}
};
```

3.2.2 write_single_buffer_sender

단일 버퍼 쓰기를 위한 Sender입니다.

```
// category/async/io_senders.hpp
class write_single_buffer_sender
{
public:
using buffer_type = filled_write_buffer;
using result_type = result<std::reference_wrapper<buffer_type>>;
static  constexpr operation_type my_operation_type = operation_type::write;
private:
```

```
chunk_offset_t offset_;

buffer_type buffer_;

std::byte *append_; // 현재 쓰기 위치

public:

// 작업 시작

result<void> operator()(erased_connected_operation *io_state) noexcept

{

MONAD_DEBUG_ASSERT(!!buffer_);

buffer_.set_bytes_transferred(size_t(append_ - buffer_.data()));

io_state->executor()->submit_write_request(buffer_, offset_, io_state);

return  success();

}

// 완료 처리

result_type completed(

erased_connected_operation *,

result<size_t> bytes_transferred) noexcept

{

BOOST_OUTCOME_TRY(auto &&count, std::move(bytes_transferred));

buffer_.set_bytes_transferred(count);

return std::ref(buffer_);

}

// 버퍼에 데이터 추가

constexpr std::byte *advance_buffer_append(size_t  bytes) noexcept
```

```
{
if (bytes > remaining_buffer_bytes()) {
return nullptr;
}
auto *ret = append_;
append_ += bytes;
return ret;
}
};
```

쓰기 Sender 특징:

- Append 포인터로 점진적 버퍼 채우기
- 8MB 버퍼에 여러 데이터 배치
- 한 번의 쓰기로 여러 작업 처리

3.2.3 read_multiple_buffer_sender

Scatter 읽기를 위한 Sender입니다.

```
// category/async/io_senders.hpp
class read_multiple_buffer_sender
{
public:
using buffers_type = std::span<std::span<std::byte>>;
using result_type = result<buffers_type>;
```

```
static  constexpr operation_type my_operation_type =

operation_type::read_scatter;

private:

chunk_offset_t offset_;

buffers_type buffers_;

std::variant<

std::array<struct iovec, 4>,

std::vector<struct iovec>

> iovecs_; // iovec 배열 (작은 경우 스택, 큰 경우 힙)

public:

result<void> operator()(erased_connected_operation *io_state) noexcept

{

// iovec 배열 준비

std::span<const  struct iovec> iovecs;

if (buffers_.size() <= 4) {

// 작은 경우 스택 사용

std::array<struct iovec, 4> temp;

for (size_t n = 0; n < buffers_.size(); n++) {

temp[n] = {

(char *)buffers_[n].data(),

buffers_[n].size()

};

}
```

```
iovecs_ = std::move(temp);

iovecs = std::get<0>(iovecs_);

} else {

// 큰 경우 힙 사용

std::vector<struct iovec> temp;

temp.reserve(buffers_.size());

for (size_t n = 0; n < buffers_.size(); n++) {

temp.push_back({

(char *)buffers_[n].data(),

buffers_[n].size()

});

}

iovecs_ = std::move(temp);

iovecs = std::get<1>(iovecs_);

}

// Scatter 읽기 제출

size_t bytes_transferred =

io_state->executor()->submit_read_request(

iovecs, offset_, io_state);

return  success();

}

};
```

Scatter 읽기 최적화:

- 여러 버퍼를 한 번의 시스템 콜로 읽기
- 작은 경우 스택, 큰 경우 힙 사용
- 메모리 할당 최소화

3.3 Connected Operation

Sender와 Receiver를 연결하여 실행 가능한 상태를 만듭니다.

```
// category/async/connected_operation.hpp
template <sender Sender, receiver Receiver>
class connected_operation final
: public detail::connected_operation_completed_implementation<
detail::connected_operation_storage<
erased_connected_operation, Sender, Receiver>,
Sender, Receiver>
{
public:
// 이동 불가능 (메모리 위치 고정)
connected_operation(connected_operation const  &) = delete;
connected_operation(connected_operation &&) = delete;
connected_operation &operator=(connected_operation const  &) = delete;
connected_operation &operator=(connected_operation &&) = delete;
};
```

```
// Connect 함수
template <sender Sender, receiver Receiver>
requires(compatible_sender_receiver<Sender, Receiver>)
inline connected_operation<Sender, Receiver>
connect(AsyncIO &io, Sender &&sender, Receiver &&receiver)
{
return  connected_operation<Sender, Receiver>(
io,
static_cast<Sender &&>(sender),
static_cast<Receiver &&>(receiver));
}
```

Connected Operation 특징:

1. **메모리 고정**: 제출 후 이동 불가 (io_uring 포인터 안정성)
2. **타입 소거**: 가상 함수로 다형성 구현
3. **재사용 가능**: `reset()` 호출로 재사용

3.4 타입 안전한 에러 처리

Sender/Receiver 패턴은 컴파일 타임에 타입을 검증합니다.

```
// category/async/connected_operation.hpp
// Concept 정의
template <typename Sender, typename Receiver>
concept void_taking_sender_receiver_pair =
```

```
sender<Sender> && receiver<Receiver> &&
requires(Receiver r, erased_connected_operation *o, result<void> res) {
r.set_value(o, std::move(res));
};


template <typename Sender, typename Receiver>
concept filled_read_buffer_taking_sender_receiver_pair =
sender<Sender> && receiver<Receiver> &&
requires(
Receiver r, erased_connected_operation *o,
result<std::reference_wrapper<filled_read_buffer>> res) {
r.set_value(o, std::move(res));
};
```

타입 안전성:

- 컴파일 타임에 Sender와 Receiver 호환성 검증
- 잘못된 타입 조합 시 컴파일 에러
- 런타임 오버헤드 없음

3.5 비동기 작업 체인

Sender/Receiver 패턴을 통해 비동기 작업을 체인으로 연결할 수 있습니다.

```
// 사용 예시
auto sender = read_single_buffer_sender(offset, bytes);
```

```
auto receiver = [](erased_connected_operation *op,
result<std::reference_wrapper<filled_read_buffer>> res) {
if (res) {
auto &buffer = res.assume_value().get();
// 읽은 데이터 처리
process_data(buffer);
} else {
// 에러 처리
handle_error(res.assume_error());
}
};

// 연결 및 시작
auto connected = io.make_connected(
std::move(sender),
std::move(receiver));
connected->start();
```

작업 체인 장점:

1. **조합 가능**: 여러 작업을 순차적으로 연결
2. **에러 전파**: 에러가 자동으로 체인을 따라 전파
3. **취소 가능**: 체인 중간에 취소 가능

4. **타입 안전**: 컴파일 타임에 타입 검증

3.6 AsyncIO::make_connected()

AsyncIO는 내부 메모리에 Connected Operation을 생성합니다.

```
// category/async/io.hpp

template <sender Sender, receiver Receiver>

requires(

(Sender::my_operation_type == operation_type::read ||

Sender::my_operation_type == operation_type::write))

auto  make_connected(Sender &&sender, Receiver &&receiver)

{

return make_connected_impl_ < Sender::my_operation_type ==

operation_type::write > ([&] {

return  connect<Sender, Receiver>(

*this, std::move(sender), std::move(receiver));

});

}
```

내부 메모리 할당:

- Connected Operation을 내부 풀에서 할당
- 쓰기 작업은 8MB 버퍼와 함께 할당
- 읽기 작업은 나중에 버퍼 할당
- RAII로 자동 해제

4. 스토리지 풀 관리 분석

4.1 Storage Pool 구조

Storage Pool은 여러 디바이스를 통합하여 청크 기반 스토리지를 제공합니다.

```
// category/async/storage_pool.hpp

class storage_pool

{

public:

enum chunk_type {

cnv, // Conventional (랜덤 읽기/쓰기)

seq  // Sequential (순차 쓰기)

};

private:

std::vector<device> devices_; // 디바이스 목록

std::vector<chunk_info> chunks_[2]; // cnv, seq 청크

mutable std::mutex lock_; // 동시성 제어

bool is_read_only_;

bool is_newly_truncated_;

public:

// 청크 활성화

std::shared_ptr<class chunk> activate_chunk(
```

```
chunk_type which,
uint32_t  id);
// 청크 개수
size_t  chunks(chunk_type which) const  noexcept
{
return  chunks_[which].size();
}
// 현재 활성 청크 수
size_t  currently_active_chunks(chunk_type which) const  noexcept;
};
```

청크 타입:

- **Conventional (cnv)**: 랜덤 읽기/쓰기 가능, 메타데이터용
- **Sequential (seq)**: 순차 쓰기만 가능, 대용량 데이터용

4.2 Device 추상화

Storage Pool은 파일, 블록 디바이스, Zoned 디바이스를 지원합니다.

```
// category/async/storage_pool.hpp
class storage_pool::device
{
public:
enum  class type_t_ {
file, // 일반 파일
```

```
block_device, // 블록 디바이스 (SSD, NVMe)
zoned_device  // Zoned 스토리지 (ZNS SSD)
};
private:
int readwritefd_; // 파일 디스크립터
type_t_ type_; // 디바이스 타입
uint64_t unique_hash_; // 고유 해시
size_t size_of_file_; // 파일 크기
metadata_t *metadata_; // 메타데이터
public:
bool is_file() const noexcept { return type_ == type_t_::file; }
bool is_block_device() const noexcept { return type_ == type_t_::block_device;
bool is_zoned_device() const noexcept { return type_ == type_t_::zoned_device;
// 청크 개수
size_t chunks() const;
// 용량 정보
std::pair<file_offset_t, file_offset_t> capacity() const;
};
```

디바이스 타입별 최적화:

- 파일: `fallocate` 로 공간 할당
- 블록 디바이스: `BLKDISCARD` 로 TRIM
- Zoned 디바이스: Zone 리셋 (미구현)

4.3 Chunk 관리

Chunk는 고정 크기의 스토리지 단위입니다.

```
// category/async/storage_pool.hpp

class storage_pool::chunk

{

protected:

device const &device_; // 소속 디바이스

int read_fd_; // 읽기 파일 디스크립터

int write_fd_; // 쓰기 파일 디스크립터

file_offset_t offset_; // 디바이스 내 오프셋

file_offset_t capacity_; // 청크 용량

uint32_t chunkid_within_device_; // 디바이스 내 청크 ID

bool owns_readfd_;

bool owns_writefd_;

bool append_only_; // 순차 쓰기 전용

public:

// 읽기 파일 디스크립터

std::pair<int, file_offset_t> read_fd() const  noexcept

{

return {read_fd_, offset_};

}

// 쓰기 파일 디스크립터 (append_only면 현재 위치 반환)
```

```
std::pair<int, file_offset_t> write_fd(
size_t  bytes_which_shall_be_written) noexcept;
// 청크 크기
file_offset_t size() const;
// 청크 내용 삭제
void  destroy_contents();
// 청크 내용 복사
uint32_t  clone_contents_into(chunk &other, uint32_t  bytes);
// 청크 내용 TRIM
bool  try_trim_contents(uint32_t  bytes);
};
```

Chunk 특징:

- 기본 크기: 4MB (설정 가능)
- Append-Only: 순차 쓰기로 SSD 수명 연장
- TRIM 지원: 사용하지 않는 공간 해제

4.4 청크 할당 및 해제

```
// category/async/storage_pool.cpp
std::shared_ptr<class storage_pool::chunk>
storage_pool::activate_chunk(chunk_type const which, uint32_t const id)
{
std::unique_lock g(lock_);
```

```
// 이미 활성화된 청크 확인
auto ret = chunks_[which][id].chunk.lock();
if (ret) {
return ret;
}
g.unlock();
auto &chunkinfo = chunks_[which][id];
// 청크 타입에 따라 생성
switch (which) {
case chunk_type::cnv:
ret = std::shared_ptr<cnv_chunk>(new cnv_chunk(
chunkinfo.device,
chunkinfo.device.readwritefd_,
chunkinfo.device.readwritefd_,
file_offset_t(chunkinfo.chunk_offset_into_device) *
chunkinfo.device.metadata_->chunk_capacity,
chunkinfo.device.metadata_->chunk_capacity,
chunkinfo.chunk_offset_into_device,
id,
false, false, false));
break;
case chunk_type::seq:
ret = std::shared_ptr<seq_chunk>(new seq_chunk(
```

```
chunkinfo.device,
chunkinfo.device.readwritefd_,
chunkinfo.device.readwritefd_,
file_offset_t(chunkinfo.chunk_offset_into_device) *
chunkinfo.device.metadata_->chunk_capacity,
chunkinfo.device.metadata_->chunk_capacity,
chunkinfo.chunk_offset_into_device,
id,
false, false, true)); // append_only = true
break;
}
g.lock();
// 다른 스레드가 이미 생성했는지 확인
auto  const ret2 = chunks_[which][id].chunk.lock();
if (ret2) {
return ret2;
}
chunks_[which][id].chunk = ret;
return ret;
}
```

청크 활성화 전략:

- Lazy 활성화: 필요할 때만 생성

- Weak 포인터: 사용하지 않으면 자동 해제
- Double-checked locking: 동시성 제어

4.5 파일 디스크립터 등록

Storage Pool은 io_uring에 파일 디스크립터를 사전 등록합니다.

```
// category/async/io.cpp (AsyncIO 생성자)
AsyncIO::AsyncIO(class storage_pool &pool, monad::io::Buffers &rwbuf)
{
// 청크 활성화
cnv_chunk_ = pool.activate_chunk(storage_pool::cnv, 0);
auto count = pool.chunks(storage_pool::seq);
seq_chunks_.reserve(count);
// 파일 디스크립터 수집
std::vector<int> fds;
fds.reserve(count * 2 + 2);
fds.push_back(cnv_chunk_.io_uring_read_fd);
fds.push_back(cnv_chunk_.io_uring_write_fd);
for (size_t n = 0; n < count; n++) {
seq_chunks_.emplace_back(
pool.activate_chunk(storage_pool::seq, n));
fds.push_back(seq_chunks_[n].io_uring_read_fd);
fds.push_back(seq_chunks_[n].io_uring_write_fd);
}
```

```
// 중복 제거 (같은 fd가 여러 번 나올 수 있음)
unordered_dense_map<int, int> fd_to_iouring_map;
for (auto fd : fds) {
fd_to_iouring_map[fd] = -1;
}
int idx = 0;
fds.clear();
for (auto &fd : fd_to_iouring_map) {
fd.second = idx++;
fds.push_back(fd.first);
}
// io_uring에 파일 등록
auto e = io_uring_register_files(
&uring_.get_ring(),
fds.data(),
static_cast<unsigned  int>(fds.size()));
MONAD_ASSERT(!e);
if (wr_uring_ != nullptr) {
e = io_uring_register_files(
&wr_uring_->get_ring(),
fds.data(),
static_cast<unsigned  int>(fds.size()));
MONAD_ASSERT(!e);
```

```
}

}
```

파일 등록의 이점:

1. **빠른 접근**: 파일 디스크립터 조회 오버헤드 제거
2. **고정 인덱스**: 정수 인덱스로 파일 참조
3. **중복 제거**: 같은 파일을 여러 번 등록하지 않음

4.6 청크 인터리빙

여러 디바이스의 청크를 균등하게 분산합니다.

```
// category/async/storage_pool.cpp

void storage_pool::fill_chunks_(creation_flags const  &flags)

{

if (flags.interleave_chunks_evenly) {

// 비율 계산 (예: 디바이스 A=20, B=10, C=5 → 4:2:1)

std::vector<double> chunkratios(chunks.size());

std::vector<double> chunkcounts(chunks.size());

for (size_t n = 0; n < chunks.size(); n++) {

chunkratios[n] = double(total) / static_cast<double>(chunks[n]);

chunkcounts[n] = chunkratios[n];

}

// 인터리빙 (ABACABA 패턴)

while (chunks_[seq].size() < chunks_[seq].capacity()) {
```

```
for (size_t n = 0; n < chunks.size(); n++) {
chunkcounts[n] -= 1.0;
if (chunkcounts[n] < 0) {
chunks_[seq].emplace_back(
std::weak_ptr<class chunk>{},
devices_[n],
chunks[n]++);
chunkcounts[n] += chunkratios[n];
}
}
}
}
}
```

인터리빙의 장점:

- 부하 분산: 여러 디바이스에 I/O 분산
- 병렬성: 동시에 여러 디바이스 접근
- 성능 향상: 단일 디바이스 병목 방지

4.7 메타데이터 관리

각 디바이스는 메타데이터를 파일 끝에 저장합니다.

```
// category/async/storage_pool.hpp
struct device::metadata_t
```

```
{
char  magic[4]; // "MND0"
uint32_t chunk_capacity; // 청크 크기
uint32_t config_hash; // 설정 해시
// 청크 사용량 배열 (파일 끝에서 역순)
std::atomic<uint32_t> *chunk_bytes_used(size_t  file_size) const
{
auto  const chunks_count = chunks(file_size);
return const_cast<std::atomic<uint32_t> *>(
reinterpret_cast<std::atomic<uint32_t> const *>(
reinterpret_cast<std::byte const *>(this) -
chunks_count * sizeof(uint32_t)));
}
size_t  total_size(size_t  file_size) const
{
return sizeof(metadata_t) +
round_up_align<DISK_PAGE_BITS>(
chunks(file_size) * sizeof(uint32_t));
}
};
```

메타데이터 레이아웃:

```
┌──────────────────────────────────────────────────────────┐
| Data Chunks |
| |
├──────────────────────────────────────────────────────────┤
| Chunk 0 Usage (4 bytes, atomic) |
| Chunk 1 Usage (4 bytes, atomic) |
| ... |
| Chunk N Usage (4 bytes, atomic) |
├──────────────────────────────────────────────────────────┤
| metadata_t (magic, chunk_capacity, config_hash) |
└──────────────────────────────────────────────────────────┘
```

메타데이터 특징:

- 파일 끝에 저장: 데이터와 분리
- 원자적 업데이트: `std::atomic` 으로 동시성 제어
- mmap 사용: 메모리 매핑으로 빠른 접근

---

5. 성능 최적화 기법 요약

5.1 io_uring 최적화

| 기법 | 설명 | 효과 |

|------|------|------|

| Fixed Files | 파일 디스크립터 사전 등록 | 파일 조회 오버헤드 제거 |

| Fixed Buffers | 버퍼 사전 등록 | 제로 카피 I/O |

| Batch Submission | 여러 요청 한 번에 제출 | 시스템 콜 최소화 |

| IOPOLL | 커널 폴링 모드 | 레이턴시 감소 |

| 우선순위 | I/O 우선순위 설정 | 중요 작업 우선 처리 |

| 분리된 링 | 읽기/쓰기 링 분리 | 간섭 방지 |

5.2 버퍼 관리 최적화

| 기법 | 설명 | 효과 |

|------|------|------|

| 버퍼 풀 | 사전 할당된 버퍼 재사용 | 할당 오버헤드 제거 |

| DMA 정렬 | 페이지 크기로 정렬 | 하드웨어 가속 |

| RAII | 스마트 포인터로 자동 관리 | 메모리 누수 방지 |

| 크기 최적화 | 읽기 32KB, 쓰기 8MB | 워크로드 최적화 |

5.3 Sender/Receiver 최적화

| 기법 | 설명 | 효과 |

|------|------|------|

| 타입 안전성 | 컴파일 타임 검증 | 런타임 오류 방지 |

| 제로 오버헤드 | 가상 함수 devirtualization | 성능 손실 없음 |

| 조합 가능 | 작업 체인 구성 | 코드 재사용성 |

| 에러 전파 | 자동 에러 처리 | 견고성 향상 |

5.4 스토리지 풀 최적화

| 기법 | 설명 | 효과 |

|------|------|------|

| 청크 인터리빙 | 여러 디바이스에 분산 | 병렬 I/O |

| Append-Only | 순차 쓰기 | SSD 수명 연장 |

| TRIM 지원 | 사용하지 않는 공간 해제 | 성능 유지 |

| Lazy 활성화 | 필요할 때만 청크 생성 | 메모리 절약 |

---

6. 참고 자료

6.1 io_uring 문서

- liburing: https://github.com/axboe/liburing
- io_uring 가이드: https://kernel.dk/io_uring.pdf
- Linux 커널 문서: https://www.kernel.org/doc/html/latest/io_uring/

6.2 관련 기술

- DMA (Direct Memory Access)
- NVMe (Non-Volatile Memory Express)
- Zoned Storage (ZNS SSD)
- Sender/Receiver (C++23 std::execution)

6.3 최적화 기법

- Zero-copy I/O
- Batch processing

- Memory alignment
- Lock-free programming